DO NOW — RECALL

Half adders and full adders

Name: _____ Class: _____ Date: _____

1. When we group four neighbouring 1s on a Karnaugh map, what happens to the Boolean expression?

2. Represent −13 using 8-bit sign and magnitude.

3. Insert 40, 20, 60, 10, 30, 50 into an empty binary search tree. Draw the resulting tree.

4. Using a 5-bit two’s complement mantissa and 4-bit two’s complement exponent, calculate A − B. Show exponent alignment and mantissa subtraction. A = $0.1100_2 \times 2^3$; B = $0.1000_2 \times 2^2$.

DO NOW — CHECK / OBJECTIVES

Correct and improve

Check page 1 in GREEN PEN. Write corrections in full.

1. Model answer

The Boolean expression becomes simpler. Variables that change within the group can be removed, leaving only variables that stay the same.

2. Model answer

Step 1: 13 in binary is 1101. Step 2: pad the magnitude to seven bits: 0001101. Step 3: because the number is negative, put 1 in the sign bit. Answer: 1 | 0001101 = 10001101. This is sign and magnitude , not two’s complement. SIGN · 1 BIT 1 Negative MAGNITUDE · 7 BITS 0001101 13 in binary

3. Model answer

Insert the values in the order given . 40 becomes the root. 20 goes left of 40; 60 goes right of 40. 10 goes left of 20; 30 goes right of 20; 50 goes left of 60. Every smaller value goes to the left and every larger value to the right. 40 20 60 10 30 50

4. Model answer

Align exponents: B = $0.0100_2 \times 2^3$. Subtract mantissas: $0.1100_2 - 0.0100_2 = 0.1000_2$. Answer: $0.1000_2 \times 2^3 = 4$.

By the end, you can…

- derive the half-adder Sum and Carry expressions
- complete half-adder and full-adder truth tables
- explain how full adders connect to add multi-bit binary numbers

Vocabulary — know these terms

Recall the term — half adder

Recall the term — full adder

Recall the term — Sum

Recall the term — Carry

Recall the term — carry-in

Recall the term — carry-out

Recall the term — XOR

Recall the term — AND

New Learning · Half and full adders

Original explanation and illustration · part 1 of 4

01 · Start with binary addition

One binary column can produce a sum bit and a carry bit.

For 1 + 1, the right-hand bit is Sum = 0 ; the left-hand bit is Carry = 1 .

Binary sums: 0 + 0 = 0; 0 + 1 = 1; 1 + 0 = 1; 1 + 1 = 10_2

02 · Meet the half adder

A half adder adds two single-bit inputs: A and B.

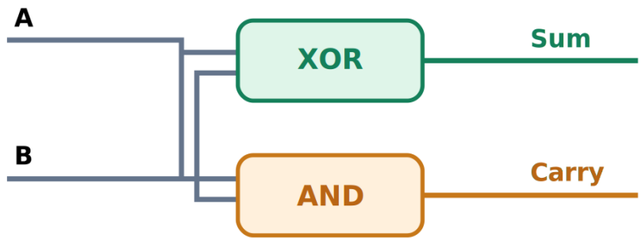


Complete five-gate full adder. A and B enter XOR and AND gates. XOR output S1 enters a second XOR and AND with Carry-in from the left. The second XOR outputs Sum; both AND carry outputs C1 and C2 enter OR and produce Carry-out. Crossed wires without junction dots are not connected.

03 · Discover the two gates

XOR calculates Sum; AND calculates Carry.

Sum = A ⊻ B (XOR) · Carry = A ∧ B



Complete five-gate full adder. A and B enter XOR and AND gates. XOR output S1 enters a second XOR and AND with Carry-in from the left. The second XOR outputs Sum; both AND carry outputs C1 and C2 enter OR and produce Carry-out. Crossed wires without junction dots are not connected.

New Learning · Half and full adders

Original explanation and illustration · part 2 of 4

04 · Build its truth table

Test every possible pair of inputs.

A | B | Sum (XOR) | Carry (AND)

0 | 0 | 0 | 0

0 | 1 | 1 | 0

1 | 0 | 1 | 0

1 | 1 | 0 | 1

05 · Find the limitation

What happens when a previous column sends a carry into this column?

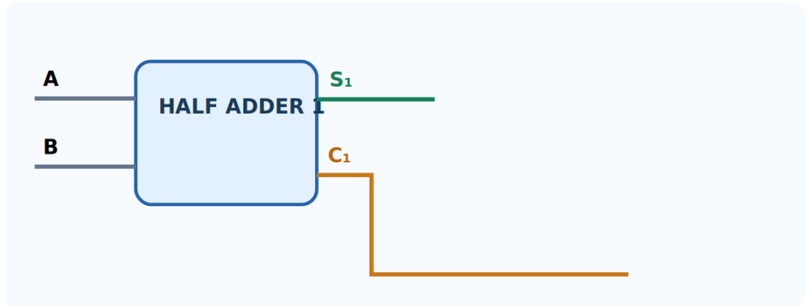
The half adder only accepts A and B . It has no third input for Carry-in (C_{in}) .

A full adder accepts A, B and C_{in}.

Binary sums: $1 + 1 + 1 = 11_2$

06 · First half adder · 1 of 3

Add A and B to produce an intermediate Sum S_1 and Carry C_1.



Complete five-gate full adder. A and B enter XOR and AND gates. XOR output S1 enters a second XOR and AND with Carry-in from the left. The second XOR outputs Sum; both AND carry outputs C1 and C2 enter OR and produce Carry-out. Crossed wires without junction dots are not connected.

EXIT TICKET / REFLECTION

Check your understanding

Complete the actual lesson Exit Ticket before checking the answer.

For a half adder, Sum is produced by XOR and Carry is produced by AND.

How confident are you? Circle one: Confident / Getting there / Need help

Explain your choice, identifying one concept you can now use independently.

GREEN PEN — Correct and explain

Choose one mistake from your independent work. Write the correct method and explain precisely why your original answer was wrong.

Next step: revisit the model or a relevant revision-guide section if needed.

YOU DO — INDEPENDENT PRACTICE

Questions · Part 2

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

Question 3

Explain four-bit addition
Explain how four full adders can add two 4-bit binary numbers.

Question 4

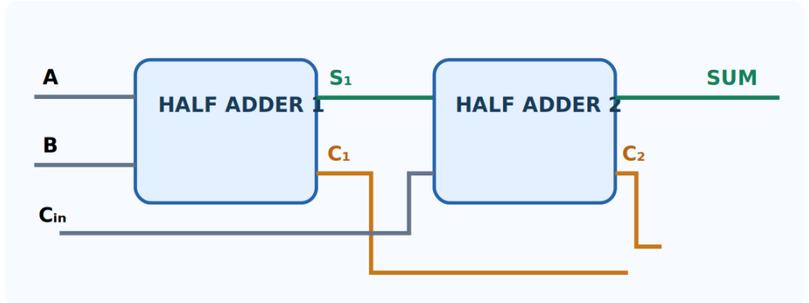
Add 0111 + 0001
Use bit-by-bit addition and identify where the carry travels.

New Learning · Half and full adders

Original explanation and illustration · part 3 of 4

07 · Second half adder · 2 of 3

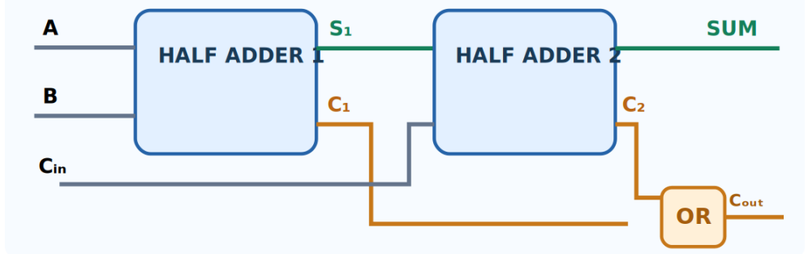
Feed S_1 and Carry-in into a second half adder. Its Sum is the final Sum.



Complete five-gate full adder. A and B enter XOR and AND gates. XOR output S1 enters a second XOR and AND with Carry-in from the left. The second XOR outputs Sum; both AND carry outputs C1 and C2 enter OR and produce Carry-out. Crossed wires without junction dots are not connected.

08 · Connect both carries · 3 of 3

Feed C_1 and C_2 into an OR gate to obtain Carry-out.

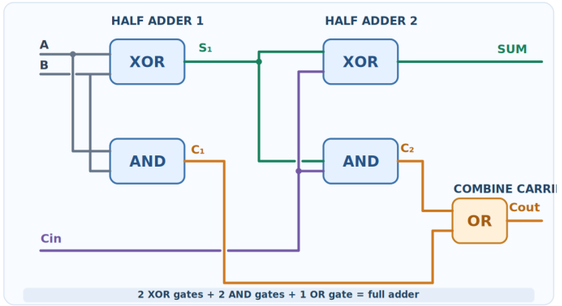


Complete five-gate full adder. A and B enter XOR and AND gates. XOR output S1 enters a second XOR and AND with Carry-in from the left. The second XOR outputs Sum; both AND carry outputs C1 and C2 enter OR and produce Carry-out. Crossed wires without junction dots are not connected.

New Learning · Half and full adders

Original explanation and illustration · part 4 of 4

09 · Full Adder — All Five Gates



Complete five-gate full adder. A and B enter XOR and AND gates. XOR output S1 enters a second XOR and AND with Carry-in from the left. The second XOR outputs Sum; both AND carry outputs C1 and C2 enter OR and produce Carry-out. Crossed wires without junction dots are not connected.

YOU DO — INDEPENDENT PRACTICE

Questions · Part 1

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

Question 1

Half-adder question
State the Sum/Carry expressions, complete the truth table, then explain how half adders contribute to a full adder.

Question 2

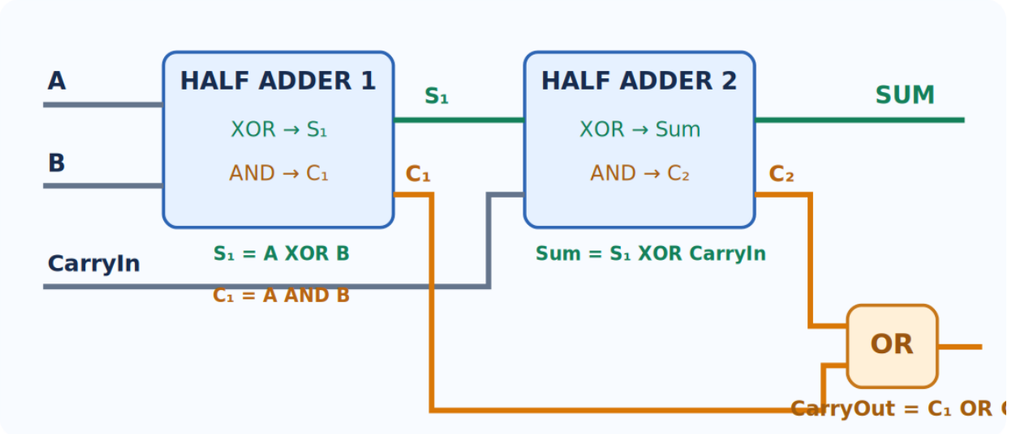
Complete eight cases
Without looking at the diagram, sketch the five gates and their connections from memory. Then create the full-adder truth table for A, B and CarryIn.

We Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

From half to full adder

Trace the two XOR gates, then the two AND gates feeding OR.



Full adder built from two half adders: A and B enter the first half adder; its sum S1 and CarryIn enter the second half adder; carry C1 and C2 are combined using an OR gate to produce CarryOut

I DO — WORKED EXAMPLE

Follow the worked method

Model 1 + 1 with a half adder

Rightmost result bit

```
Sum = 0
```

Next column

```
Carry = 1
```

Trace XOR and AND

```
1 XOR 1 = 0 → Sum
1 AND 1 = 1 → Carry
```

Connect both half adders

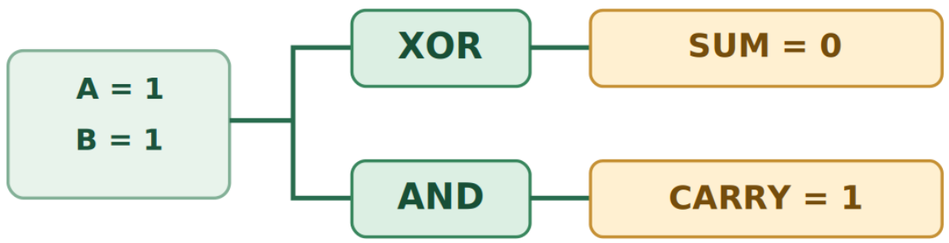
```
S₁ = A XOR B
Sum = S₁ XOR Cin
Cout = (A AND B) OR (S₁ AND Cin)
```

I Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

Model 1 + 1 with a half adder

We have just learned XOR and AND. Now model the output when A = 1 and B = 1.



1 + 1 = 10₂ → sum bit 0, carry bit 1

Half adder with inputs A and B both one; XOR produces sum zero and AND produces carry one

WE DO — GUIDED PRACTICE

Try → Discuss → Improve

1. Half-adder expressions

OCR asks for S and C.

```
S = A ? B
C = A ? B
```

Method / working:

2. Half-adder truth table

Complete the four outputs.

```
A B | S C
0 0 | ? ?
0 1 | ? ?
1 0 | ? ?
1 1 | ? ?
```

Method / working:

3. From half to full adder

Trace the two XOR gates, then the two AND gates feeding OR.

```
Two half adders plus an ? gate can form a full adder.
```

Method / working:

4. Full-adder case

A=1, B=1, CarryIn=1.

```
1 + 1 + 1 = binary ?
Sum = ?
CarryOut = ?
```

Method / working: